



Al-Imam University

Computer Science Department

College of Computer and Information Sciences

Design and Implementation of a Compiler Front-End for the HealthScript Language

Phase 1

Students:

Name	ID
Nuha Aedh Alrubayyi	444005638
Dan Riyadh Alhumrani	444006468

CONTENTS

1. Language Specification	3
2. Phase 1: Lexical Analysis	4
3. Scanner Objective	6
4. Token Definitions	7
5. Regular Expressions Used	10
6. DFA Diagrams for Token Recognition	12
7. Scanner Algorithm	14
8. Symbol Table	15
9. Error Handling	16
10. Test Cases.....	18
11. Conclusion	23
12. Appendix: Full Source Code	24

1. Language Specification

The proposed programming language, HealthScript, is a domain-specific scripting language designed to simplify programming tasks in healthcare environments. The primary goal of HealthScript is to provide a readable and easy-to-understand syntax that can be used by non-technical users such as nurses, healthcare administrators, and medical data analysts. By using plain English keywords and a simplified structure, the language reduces the complexity typically associated with programming and allows users to focus on healthcare logic rather than syntax rules.

HealthScript is particularly useful for automating routine healthcare-related tasks such as monitoring patient vital signs, generating alerts for abnormal conditions, handling simple data input and output, and ensuring that ethical requirements—such as patient consent—are checked before performing certain operations. This makes the language not only functional but also aligned with real-world healthcare practices.

The language supports a set of fundamental data types including int, float, string, and bool, which are sufficient to represent common healthcare data such as patient age, temperature readings, diagnostic notes, and consent status. Variables in HealthScript must be explicitly declared using the var keyword, ensuring clarity and reducing ambiguity in programs.

HealthScript includes essential operators such as arithmetic operators (+, -, *, /, %, ^), relational operators (==, !=, <, >, <=, >=), and logical operators (and, or, not). These operators enable users to perform calculations, comparisons, and logical decision-making within their programs. Control structures such as if-then-else, while, and repeat allow for conditional execution and iteration, making the language flexible enough to handle different scenarios.

Another key feature of HealthScript is its case-insensitive nature, meaning that keywords and identifiers are not affected by letter casing. This further simplifies usage for beginners. Identifiers must begin with a letter and can contain letters and digits, with a maximum length of eight characters, ensuring consistency and preventing overly complex naming.

The syntax of HealthScript follows a structured format where each program begins with the keyword `start` and ends with `finish`. Blocks of code, such as conditional statements and functions, are terminated using the `finish` keyword instead of traditional braces, improving readability. Additionally, the language supports both single-line comments (`//`) and block comments (`/* ... */`) to enhance code documentation.

To support the lexical analysis phase, regular expressions are defined for all token types, including identifiers, numbers, and strings. These definitions ensure that the scanner can accurately recognize and categorize different elements of the source code. Overall, HealthScript is designed to bridge the gap between healthcare professionals and programming by offering a simple, readable, and practical language tailored to real-world applications in the medical domain.

2. Phase 1: Lexical Analysis

The first phase of the compiler project focuses on Lexical Analysis, which involves building a scanner that reads the source code written in HealthScript and converts it into a sequence of tokens. This phase is essential because it forms the foundation for subsequent stages such as syntax and semantic analysis.

The scanner developed in this project was implemented from scratch using Python, without relying on any external lexer-generation tools. Its primary function is to process the input program character by character and group sequences of characters

into meaningful tokens such as keywords, identifiers, operators, literals, and delimiters.

To achieve accurate token recognition, the scanner utilizes regular expressions (RE) to define patterns for each token type. For example, identifiers follow the pattern of starting with a letter and optionally including digits, while numeric literals are categorized into integers and floating-point numbers with specific constraints. String literals are recognized as sequences of characters enclosed within double quotation marks.

The scanner is designed to be case-insensitive, meaning that all identifiers and keywords are processed in a uniform manner regardless of their letter casing. This is achieved by converting identifier values to lowercase before checking them against the predefined list of reserved keywords.

One of the key responsibilities of the lexical analyzer is to generate detailed output that includes:

- The total number of lexemes identified in the source code.
- A complete list of tokens, where each token includes its type, line number, and value.
- A symbol table that stores all identifiers along with the line number of their first occurrence.
- A list of lexical errors detected during scanning.

Error handling is an important component of this phase. The scanner is capable of detecting and reporting various types of lexical errors, including:

- Invalid symbols that do not belong to the language.
- Identifiers that exceed the maximum allowed length.
- Malformed numeric literals that violate the defined constraints.

- Unterminated string literals.
- Unterminated block comments.

Each error is reported with a clear and descriptive message along with the corresponding line number, making it easier to debug the source code.

Additionally, the scanner properly handles whitespace and comments by ignoring them during token generation, while still maintaining accurate line numbering. This ensures that the output is both clean and informative.

To validate the correctness of the scanner, multiple test cases were used, including both valid programs and programs containing deliberate lexical errors. These test cases demonstrate the scanner's ability to correctly identify tokens and detect errors as required.

In conclusion, the lexical analysis phase successfully transforms HealthScript source code into a structured sequence of tokens while providing detailed error reporting and symbol table management. This serves as a solid foundation for the next phase of the compiler, which is syntax analysis.

3. Scanner Objective

The main objective of the scanner (lexical analyzer) in this project is to read a HealthScript source program and convert it into a structured stream of tokens that can be used by the next phase of the compiler (syntax analysis). The scanner acts as the first stage of the compilation process, where raw input text is analyzed and broken down into meaningful components.

The scanner is designed to process the input program sequentially, character by character, and group sequences of characters into valid lexemes such as keywords, identifiers, literals, operators, and delimiters. Each lexeme is then classified into a corresponding token type. This transformation simplifies the input for the parser by

removing unnecessary details such as whitespace and comments while preserving the logical structure of the program.

Another important objective of the scanner is to maintain accurate tracking of line numbers throughout the scanning process. This allows the system to provide precise and meaningful error messages whenever a lexical error is detected. Line number tracking is especially important for debugging and improving code readability.

In addition to token generation, the scanner is responsible for building and maintaining a symbol table. The symbol table stores all identifiers encountered in the program along with the line number of their first occurrence. This structure is essential for later phases of the compiler, such as semantic analysis.

Error detection is also a key objective of the scanner. The implemented scanner is capable of identifying several types of lexical errors, including invalid symbols, identifiers that exceed the maximum allowed length, malformed numeric literals, and unterminated string literals or block comments. Each error is reported clearly with its corresponding line number to assist users in correcting their code.

Overall, the scanner ensures that the input program is transformed into a clean, organized, and error-checked sequence of tokens, forming a reliable foundation for further analysis in the compiler pipeline.

4. Token Definitions

In HealthScript, tokens represent the smallest meaningful units of the source code. Each token is classified based on its role in the language and is identified using regular expressions. The scanner categorizes tokens into several types, each corresponding to a specific syntactic element.

The first category is Keywords, which are reserved words that have predefined meanings in the language and cannot be used as identifiers. Examples include start,

finish, if, then, else, while, repeat, var, int, float, string, bool, read, print, func, return, and, or, and not. These keywords define the structure and behavior of HealthScript programs.

The second category is Identifiers, which represent variable names, function names, and other user-defined entities. Identifiers must begin with an English letter and can be followed by letters or digits. Additionally, the maximum allowed length for an identifier is eight characters, ensuring simplicity and consistency within the language.

The third category includes Numeric Literals, which are divided into integers and floating-point numbers. Integer literals consist of digits with an optional sign, while floating-point literals include a decimal point and digits on both sides. HealthScript enforces a constraint that numeric values must not exceed eight digits before the decimal point, and any violation of this rule is treated as a lexical error.

Another important category is String Literals, which are sequences of characters enclosed within double quotation marks. These are typically used for output messages and textual data. The scanner ensures that all string literals are properly terminated; otherwise, an error is reported.

The language also includes Operators, which are used to perform operations on values. These are divided into arithmetic operators (+, -, *, /, %, ^), relational operators (==, !=, <, >, <=, >=), and logical operators (and, or, not). The assignment operator (=) is also included for assigning values to variables.

Additionally, Delimiters are used to separate different elements of the program. These include symbols such as parentheses, commas, semicolons, and braces. Delimiters help define the structure and organization of statements and expressions.

Finally, the scanner also recognizes Comments and Whitespace, which are ignored during token generation. Comments can be either single-line (starting with //) or block

comments enclosed between `/*` and `*/`. Although these elements are not converted into tokens, they are handled carefully to maintain accurate line numbering.

Each of these token types is defined using regular expressions, enabling the scanner to efficiently and accurately classify different parts of the source code. This structured classification is essential for ensuring that the program is correctly interpreted in later stages of compilation.

Table 1 : Summary of Token Types

Token Type	Description	Example
KEYWORD	Reserved words with predefined meaning	start, if, var
IDENTIFIER	User-defined names for variables/functions	ptTemp, heartRt
INTEGER	Whole numbers (max 8 digits)	10, -5
FLOAT	Decimal numbers (max 8 digits before decimal)	36.5, -12.75
STRING	Text enclosed in double quotes	"Hello"
REL_OP	Relational operators	==, !=, >, <=
ARITH_OP	Arithmetic and assignment operators	+, -, *, /, =
DELIMITER	Symbols for structure and separation	(,), :, ,
LINE_COMMENT	Single-line comments (ignored)	// comment
BLOCK_COMMENT	Multi-line comments (ignored)	/* comment */
WHITESPACE	Spaces and tabs (ignored)	(space), tab
NEWLINE	Line breaks (used for tracking lines)	\n
UNKNOWN	Invalid or unrecognized symbols	@, #

ERROR	Invalid tokens or malformed inputs	long identifier, bad number
-------	------------------------------------	-----------------------------

5. Regular Expressions Used

Regular Expressions (RE) are used in the scanner to define patterns for recognizing different token types in HealthScript. These patterns allow the lexical analyzer to efficiently match sequences of characters in the source code and classify them into appropriate tokens.

Each token type is associated with a specific regular expression. The scanner processes the input by matching these expressions in a predefined order, ensuring that more specific patterns (such as relational operators or malformed numbers) are detected before more general ones. This ordering is critical to avoid incorrect classification.

Identifiers are defined using a pattern that ensures they start with a letter followed by letters or digits. Numeric literals are divided into integers and floating-point numbers, with constraints applied to limit the number of digits. Special patterns are also used to detect malformed numbers that exceed the allowed limits. String literals are recognized as sequences enclosed within double quotation marks, while additional patterns are included to detect unterminated strings.

Operators and delimiters are identified using character-based expressions, while comments are handled using patterns that match single-line and block comment formats. The scanner also includes a fallback pattern to detect any unknown or invalid symbols.

The following table summarizes the main regular expressions used in the scanner:

Table 2 : regular expressions

Token Type	Regular Expression	Description
BLOCK_COMMENT	<code>\/*(?::.</code>	<code>\n)*?*/^</code>
UNTERMINATED_BLOCK	<code>\/*(?::.</code>	<code>\n)*^</code>
LINE_COMMENT	<code>//[^\n]*</code>	Matches single-line comments
STRING	<code>"[^"\n]*"</code>	Matches valid string literals
UNTERMINATED_STRING	<code>"[^"\n]*</code>	Detects missing closing quotation
MALFORMED_FLOAT	<code>[+-]?d{9,}\.d+</code>	Detects floats exceeding digit limit
MALFORMED_INTEGER	<code>[+-]?d{9,}</code>	Detects integers exceeding digit limit
FLOAT	<code>[+-]?d{1,8}\.d+</code>	Matches valid floating-point numbers
INTEGER	<code>[+-]?d{1,8}</code>	Matches valid integers
REL_OP	<code>== != <= >= < ></code>	Matches relational operators
ARITH_OP	<code>[+\-/*%^=]</code>	Matches arithmetic and assignment operators
DELIMITER	<code>[.\(\),\{\}\;\:]</code>	Matches delimiters
IDENTIFIER	<code>[a-zA-Z][a-zA-Z0-9]*</code>	Matches identifiers
NEWLINE	<code>\n</code>	Tracks line breaks

WHITESPACE	<code>[\t]+</code>	Matches spaces and tabs
UNKNOWN	<code>.</code>	Matches any invalid character

These regular expressions form the backbone of the scanner, enabling it to accurately recognize tokens and detect lexical errors.

6. DFA Diagrams for Token Recognition

This section presents deterministic finite automata (DFA) used to recognize key token types in HealthScript.

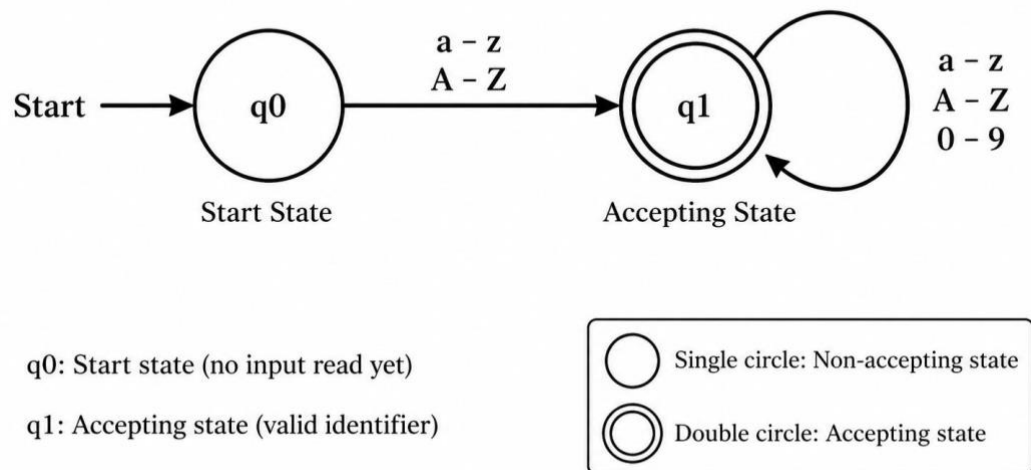


Figure 1: DFA for Identifier

This DFA recognizes identifiers that begin with a letter followed by letters or digits.

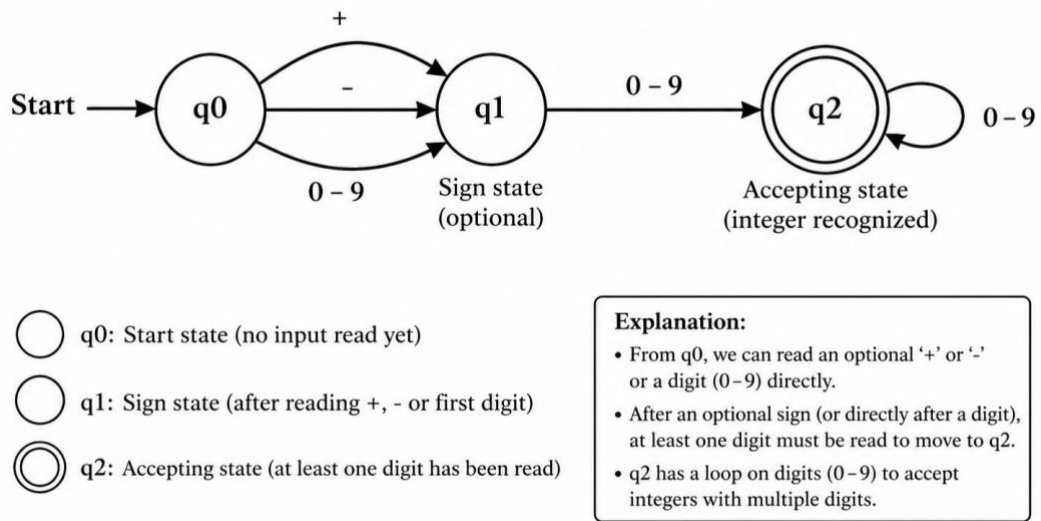


Figure 2: DFA for Integer

This DFA recognizes integer literals with an optional positive or negative sign followed by one or more digits.

DFA for String Literals

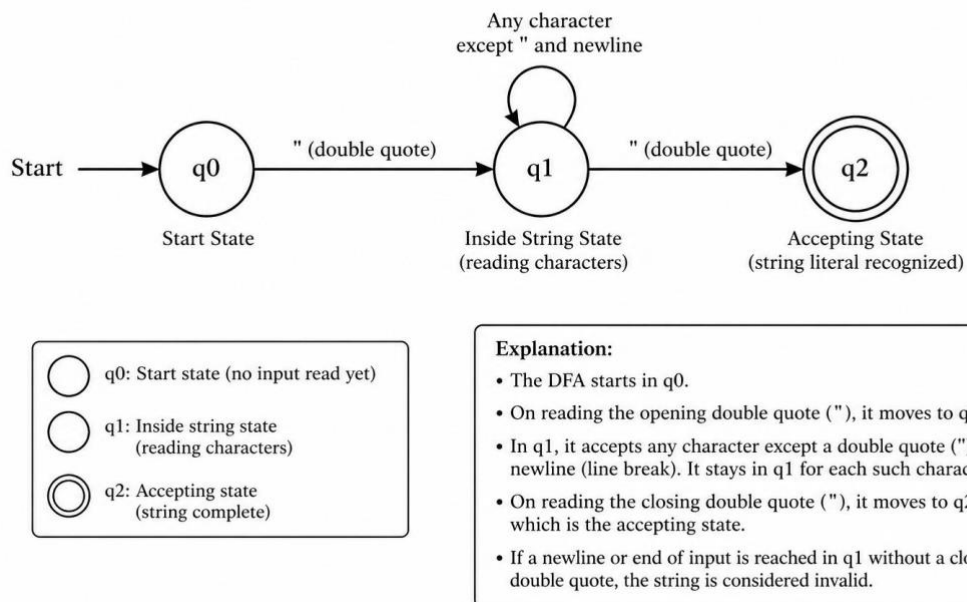


Figure 3: DFA for String Literal

This DFA is used to recognize string literals in HealthScript.

The process begins at the start state (q0). When the scanner reads an opening double quote ("), it transitions to state (q1), which represents being inside a string. In state (q1), the DFA accepts any character except a newline or another double quote. It remains in this state as long as valid characters are read. When a closing double quote (") is encountered, the DFA moves to the accepting state (q2), indicating that a valid string literal has been recognized. If the input ends or a newline is reached before a closing double quote is found, the string is considered invalid and results in a lexical error.

7. Scanner Algorithm

The scanner algorithm is designed to process the input source code sequentially and convert it into a list of tokens while detecting any lexical errors. The implementation follows a structured approach that ensures correctness, efficiency, and clarity.

The algorithm begins by initializing key data structures, including a list to store tokens, a symbol table to track identifiers, a list for errors, and a counter for the total number of lexemes. A predefined set of keywords is also initialized to distinguish reserved words from identifiers.

Next, all regular expressions are combined into a single pattern using named groups. This allows the scanner to match multiple token types in a single pass through the input. The program then iterates over the source code using pattern matching.

During each iteration, the scanner identifies the type of token (token kind) and its corresponding value. If the token is a newline character, the line counter is incremented. If the token is whitespace or a comment, it is ignored. For block comments, the scanner also counts any newlines within the comment to maintain accurate line numbering.

If the matched token corresponds to an error condition such as an unterminated string, malformed number, or unknown symbol the scanner records an appropriate error message along with the line number and continues processing the rest of the input.

When a valid token is detected, the scanner checks whether it is an identifier. If so, it converts the value to lowercase and compares it with the list of keywords. If it matches a keyword, it is classified accordingly. Otherwise, it is treated as an identifier, provided it does not exceed the maximum length. Valid identifiers are added to the symbol table if they appear for the first time.

For all other token types, the scanner directly appends the token (including its type, line number, and value) to the token list. The total number of lexemes is incremented for each valid token identified.

After processing the entire input, the scanner returns the total number of lexemes, the complete token list, the symbol table, and the list of errors.

In summary, the scanner algorithm efficiently transforms raw source code into a structured representation while ensuring accurate error detection and reporting. This structured output is essential for the next phase of the compiler, which performs syntax analysis.

8. Symbol Table

The Symbol Table is a fundamental data structure used in the lexical analysis phase to store information about identifiers encountered in the source code. In this project, the symbol table is implemented as a dictionary where each identifier is stored along with the line number of its first occurrence.

During the scanning process, whenever an identifier is recognized and validated (i.e., it meets the naming rules and does not exceed the maximum allowed length), the scanner checks whether it already exists in the symbol table. If the identifier appears

for the first time, it is added to the table along with the current line number. If it has already been recorded, no duplication occurs, ensuring that each identifier is stored only once.

The symbol table serves several important purposes. First, it provides a clear overview of all user-defined names used in the program, which helps in understanding the structure of the code. Second, it plays a critical role in later phases of the compiler, especially semantic analysis, where information such as variable declarations, scope, and type checking will be required.

For example, in a sample HealthScript program, identifiers such as `ptTemp`, `heartRt`, and `errInt` are stored in the symbol table along with the line numbers where they first appear. This allows the compiler to reference these identifiers efficiently during subsequent processing.

The design of the symbol table in this phase is kept simple, focusing only on storing the identifier name and its first occurrence. However, it can be extended in future phases to include additional attributes such as data type, scope level, and memory location.

Overall, the symbol table enhances the organization and management of identifiers, making it an essential component of the lexical analyzer.

9. Error Handling

Error handling is a crucial component of the lexical analysis phase, as it ensures that any invalid or improperly formatted elements in the source code are detected and reported clearly. The HealthScript scanner is designed to identify different types of lexical errors while continuing the scanning process to capture all possible issues in a single run.

One of the main errors detected by the scanner is the presence of invalid symbols. Any character that does not match the defined token patterns is classified as an unknown token and reported as a lexical error. This prevents unsupported or unintended characters from being processed further.

The scanner also checks for identifier length violations. According to the language rules, identifiers must not exceed eight characters. If a longer identifier is encountered, the scanner reports it as an error while still continuing the analysis of the remaining code.

Another important type of error is related to malformed numeric literals. The scanner validates both integers and floating-point numbers to ensure they follow the specified constraints. If a number exceeds the allowed digit limit or does not match the correct format, it is reported as a malformed number.

In addition, the scanner detects unterminated string literals, which occur when a string begins with a quotation mark but is not properly closed. This type of error is clearly reported along with the partial string value to help identify the issue.

The scanner also identifies unterminated block comments, where a comment starts with `/*` but does not end with `*/`. This ensures that incomplete comments do not affect the parsing of the remaining code.

Each detected error is stored in a dedicated error list and reported with a descriptive message and the exact line number where it occurred. This allows users to quickly locate and correct issues in their code.

Importantly, the scanner is designed to continue processing even after encountering errors, rather than stopping execution. This approach improves usability by allowing multiple errors to be identified in a single run.

Overall, the error handling mechanism enhances the reliability of the scanner by ensuring that all lexical issues are detected, clearly reported, and easy to debug.

10. Test Cases

Test Case 1 : Valid Program

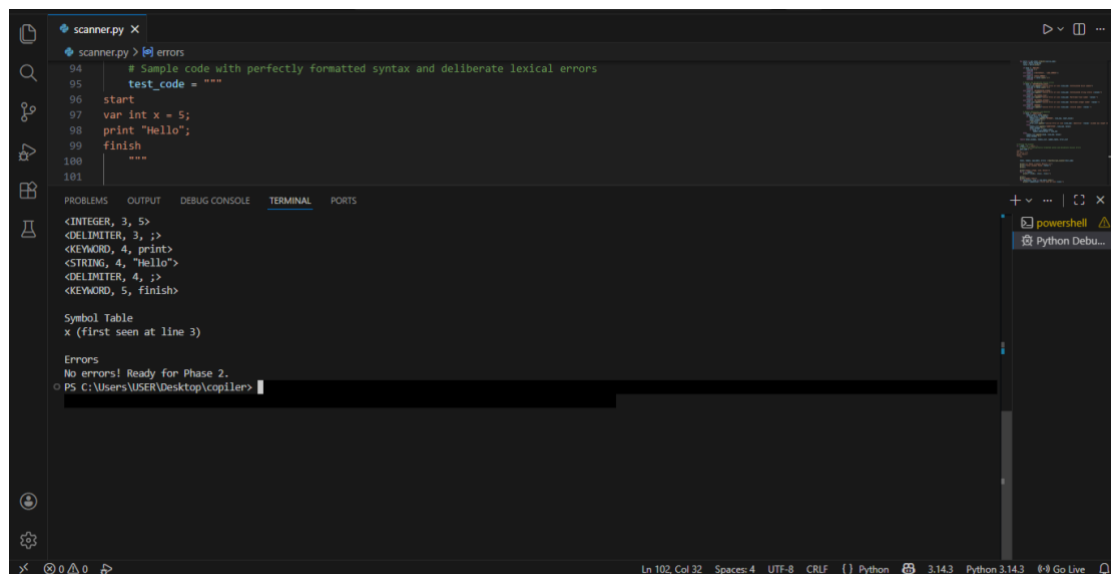
Code

start

var int x = 5;

print "Hello";

finish



Output:

The scanner successfully analyzed the program. All tokens were correctly identified, the symbol table was generated, and no lexical errors were found.

Test Case 2 : Valid Program

Code

start

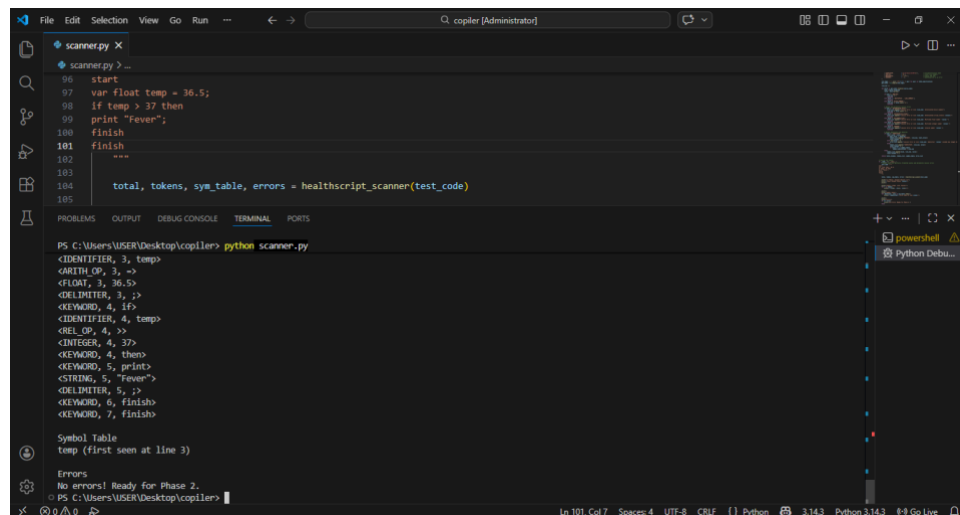
var float temp = 36.5;

if temp > 37 then

print "Fever";

finish

finish



The screenshot shows a VS Code editor window with a file named `scanner.py`. The code in the editor is as follows:

```
96 start
97 var float temp = 36.5;
98 if temp > 37 then
99   print "Fever";
100 finish
101 finish
102 ---
103
104 total, tokens, sym_table, errors = healthscript_scanner(test_code)
105
```

The terminal output shows the execution of the script:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 3, temp>
<RETH OP, 3, =>
<FLOAT, 3, 36.5>
<DELIMITER, 3, ;>
<KEYWORD, 4, if>
<IDENTIFIER, 4, temp>
<REL OP, 4, >>
<INTEGER, 4, 37>
<KEYWORD, 4, then>
<KEYWORD, 5, print>
<STRING, 5, "Fever">
<DELIMITER, 5, ;>
<KEYWORD, 6, finish>
<KEYWORD, 7, finish>

Symbol Table
temp (first seen at line 3)

Errors
No errors! Ready for Phase 2.
PS C:\Users\USER\Desktop\copiler>
```

Output:

The scanner successfully processed the input program. All tokens were recognized correctly, and no lexical errors were detected.

Test Case 3 : Valid Program

Code

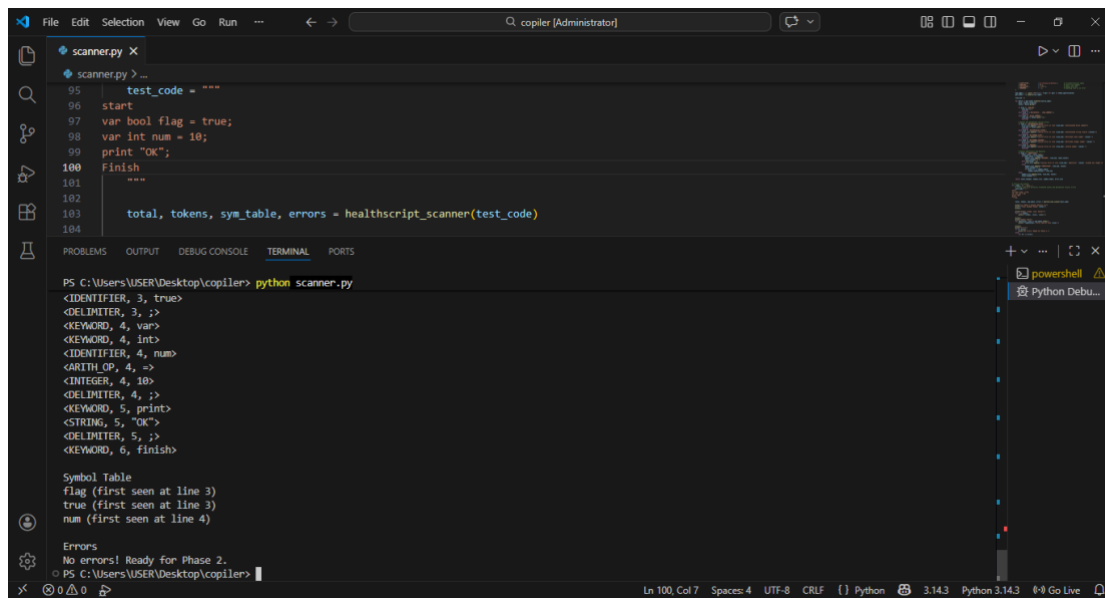
start

var bool flag = true;

var int num = 10;

print "OK";

finish



The screenshot shows a VS Code editor window with a file named `scanner.py`. The code in the editor is as follows:

```
95 test_code = """
96 start
97 var bool flag = true;
98 var int num = 10;
99 print "OK";
100 Finish
101 """
102
103 total, tokens, sym_table, errors = healthscript_scanner(test_code)
104
```

The terminal output shows the execution of the script, displaying the tokens and symbol table entries:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 3, true>
<DELIMITER, 3, ;>
<KEYWORD, 4, var>
<KEYWORD, 4, int>
<IDENTIFIER, 4, num>
<ARITHOP, 4, =>
<INTEGER, 4, 10>
<DELIMITER, 4, ;>
<KEYWORD, 5, print>
<STRING, 5, "OK">
<DELIMITER, 5, ;>
<KEYWORD, 6, finish>

Symbol Table
flag (first seen at line 3)
true (first seen at line 3)
num (first seen at line 4)

Errors
No errors! Ready for Phase 2.
PS C:\Users\USER\Desktop\copiler>
```

Output:

The scanner correctly generated tokens and symbol table entries. No lexical errors were found.

Test Case 1: Program with Lexical Errors

Code

start

```
var int veryLongVariableName = 10;
```

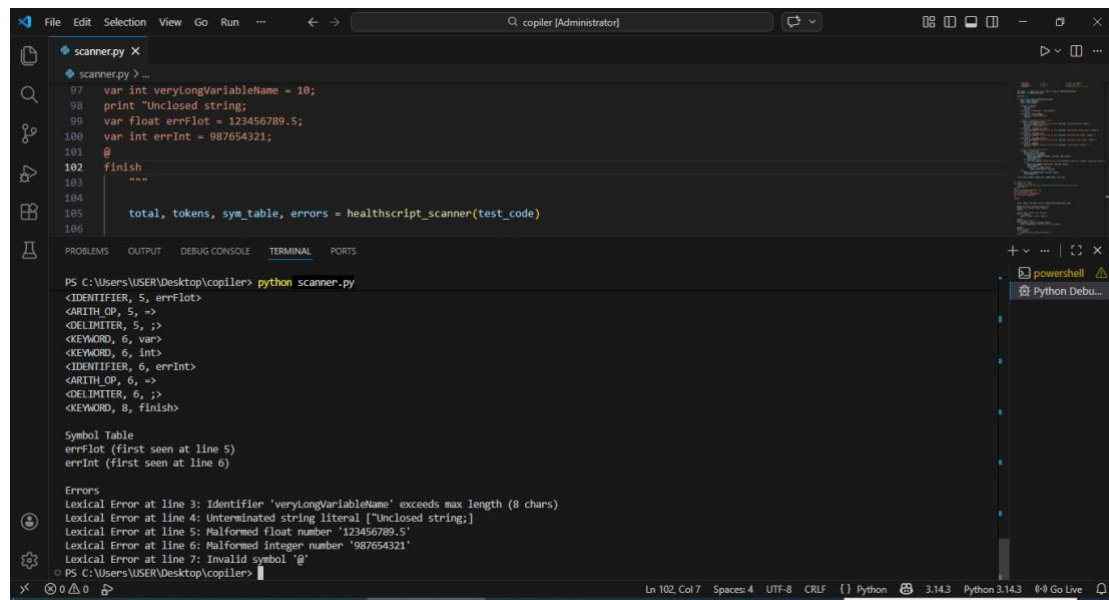
```
print "Unclosed string;
```

```
var float errFlot = 123456789.5;
```

```
var int errInt = 987654321;
```

@

finish



```
scanner.py > ...
97 var int veryLongVariableName = 10;
98 print "Unclosed string;
99 var float errFlot = 123456789.5;
100 var int errInt = 987654321;
101 @
102 finish
103 ***
104
105 total, tokens, sym_table, errors = healthscript_scanner(test_code)
106
```

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
<IDENTIFIER, 5, errFlot>
<ARITH_OP, 5, =>
<DELIMITER, 5, ;>
<KEYWORD, 6, var>
<KEYWORD, 6, int>
<IDENTIFIER, 6, errInt>
<ARITH_OP, 6, =>
<DELIMITER, 6, ;>
<KEYWORD, 8, finish>

Symbol Table
errFlot (first seen at line 5)
errInt (first seen at line 6)

Errors
Lexical Error at line 3: Identifier 'veryLongVariableName' exceeds max length (8 chars)
Lexical Error at line 4: Unterminated string literal ['Unclosed string;]
Lexical Error at line 5: Malformed float number '123456789.5'
Lexical Error at line 6: Malformed integer number '987654321'
Lexical Error at line 7: Invalid symbol '@'
```

Output:

The scanner detected multiple lexical errors, including:

- Identifier exceeding maximum length
- Unterminated string literal
- Malformed float number
- Malformed integer number
- Invalid symbol

Test Case 2 : Program with Lexical Errors

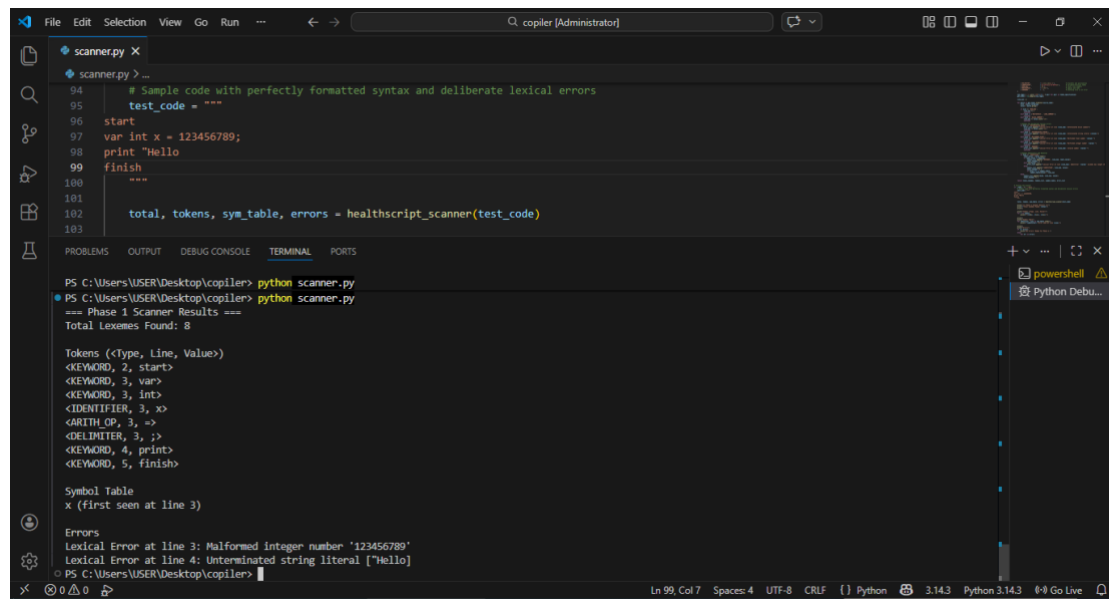
Code

start

var int x = 123456789;

print "Hello

finish



The screenshot shows a VS Code editor window with a file named 'scanner.py'. The code in the editor is as follows:

```
94 # Sample code with perfectly formatted syntax and deliberate lexical errors
95 test_code = """
96 start
97 var int x = 123456789;
98 print "Hello
99 finish
100 """
101
102 total, tokens, sym_table, errors = healthscript_scanner(test_code)
103
```

The terminal output shows the execution of the script:

```
PS C:\Users\USER\Desktop\copiler> python scanner.py
PS C:\Users\USER\Desktop\copiler> python scanner.py
=== Phase 1 Scanner Results ===
Total Lexemes Found: 8

Tokens (<Type, Line, Value>)
<KEYWORD, 2, start>
<KEYWORD, 3, var>
<KEYWORD, 3, int>
<IDENTIFIER, 3, x>
<ARITH_OP, 3, =>
<DELIMITER, 3, ;>
<KEYWORD, 4, print>
<KEYWORD, 5, finish>

Symbol Table
x (first seen at line 3)

Errors
Lexical Error at line 3: Malformed integer number '123456789'
Lexical Error at line 4: Unterminated string literal ["Hello"]
PS C:\Users\USER\Desktop\copiler>
```

Output:

The scanner reported lexical errors, including:

- Malformed integer number
- Unterminated string literal

11. Conclusion

In this phase of the project, a lexical analyzer (scanner) for the HealthScript language was successfully designed and implemented. The scanner is capable of reading source code and transforming it into a structured stream of tokens, which serves as the foundation for the subsequent phases of the compiler.

The implementation supports all required token types, including keywords, identifiers, numeric literals, string literals, operators, and delimiters. In addition, the scanner effectively ignores comments and whitespace while maintaining accurate line numbering, ensuring clean and meaningful output.

A symbol table was also constructed to store identifiers along with the line numbers of their first occurrence. This structure will be essential in later stages of the compiler, particularly in semantic analysis.

Furthermore, the scanner demonstrates strong error detection capabilities. It successfully identifies invalid symbols, identifiers exceeding the allowed length, malformed numeric values, and unterminated strings or comments. All errors are reported with clear messages and corresponding line numbers, making debugging easier.

The correctness and reliability of the scanner were verified through multiple test cases, including both valid programs and programs containing deliberate lexical errors. The results confirm that the scanner meets all the requirements specified for Phase 1.

Overall, this phase provides a solid and reliable foundation for the next stage of the project, which involves syntax analysis. The structured token output generated by the scanner will be used as input for building the parser in Phase 2.

12. Appendix: Full Source Code

This section provides the complete source code of the HealthScript lexical analyzer (scanner). The scanner is implemented in Python and was developed from scratch without using any external lexer-generation tools, in accordance with the project requirements.

The program is designed to read HealthScript source code and convert it into a sequence of tokens while generating a symbol table and detecting lexical errors. It utilizes regular expressions to define token patterns and processes the input sequentially to ensure accurate classification of all elements.

The scanner also includes a testing section that allows different input programs to be executed and analyzed. This helps demonstrate the correctness of the implementation and provides clear output for evaluation purposes.

The main components of the source code include:

- Definition of keywords and token types
- Regular expression patterns for token recognition
- Tokenization logic using pattern matching
- Symbol table construction
- Error detection and reporting
- Test case execution and output display

The full source code of the scanner is provided below:

```
import re

def healthscript_scanner(source_code):
    tokens_list = []
    symbol_table = {}
    error_list = []
    total_lexemes = 0
```

```

# Keywords we agreed on in the Language Spec document
keywords = {
    "start", "finish", "if", "then", "else", "repeat", "while",
"var",
    "int", "float", "string", "bool", "do", "read", "print",
"void",
    "return", "func", "and", "or", "not"
}

# Regular expressions for our tokens (Order is critical)
token_specification = [
    ('BLOCK_COMMENT',      r'\/*(?:.|\\n)*?\/'),          # /*
block comment */
    ('UNTERMINATED_BLOCK', r'\/*(?:.|\\n)*'),              # catch
missing */
    ('LINE_COMMENT',       r'\/\/[^\n]*'),                 # //
single line comment
    ('STRING',             r'"[^\n]*"'),                   # "text
inside quotes"
    ('UNTERMINATED_STRING', r'"[^\n]*'),                   # catch
missing end quote
    ('MALFORMED_FLOAT',    r'[+-]?\\d{9,}\\.|\\d+'),       #
Error: Float exceeding 8 digits
    ('MALFORMED_INTEGER',  r'[+-]?\\d{9,}'),               #
Error: Integer exceeding 8 digits
    ('FLOAT',              r'[+-]?\\d{1,8}\\.|\\d+'),       #
numbers with decimals (max 8)
    ('INTEGER',            r'[+-]?\\d{1,8}'),               #
normal numbers (max 8)
    ('REL_OP',             r'==|!=|<=|>=|<|>'),           #
relational operators
    ('ARITH_OP',           r'[+\\-\\*\\/\\^=]'),            # math
and assignment (=)
    ('DELIMITER',         r'[.\\(\\),\\{\\}\\};:]'),         #
brackets and punctuation
    ('IDENTIFIER',        r'[a-zA-Z][a-zA-Z0-9]*'),        #
variable/function names
    ('NEWLINE',           r'\\n'),                          # count
line numbers
    ('WHITESPACE',        r'[ \\t]+'),                      #
spaces and tabs
    ('UNKNOWN',           r'\\.'),                          #
anything else is an error
]

tok_regex = '|'.join('(P<%s>%s)' % pair for pair in
token_specification)
get_token = re.compile(tok_regex)

```

```

line_num = 1

for match in get_token.finditer(source_code):
    kind = match.lastgroup
    value = match.group()

    if kind == 'NEWLINE':
        line_num += 1
        continue
    elif kind in ['WHITESPACE', 'LINE_COMMENT']:
        continue
    elif kind == 'BLOCK_COMMENT':
        line_num += value.count('\n')
        continue

    # Handle the deliberate lexical errors
    if kind == 'UNTERMINATED_BLOCK':
        error_list.append(f"Lexical Error at line {line_num}:
Unterminated block comment")
        line_num += value.count('\n')
        continue
    elif kind == 'UNTERMINATED_STRING':
        error_list.append(f"Lexical Error at line {line_num}:
Unterminated string literal [{value}]")
        continue
    elif kind == 'MALFORMED_FLOAT':
        error_list.append(f"Lexical Error at line {line_num}:
Malformed float number '{value}'")
        continue
    elif kind == 'MALFORMED_INTEGER':
        error_list.append(f"Lexical Error at line {line_num}:
Malformed integer number '{value}'")
        continue
    elif kind == 'UNKNOWN':
        error_list.append(f"Lexical Error at line {line_num}:
Invalid symbol '{value}'")
        continue

    # Handle Identifiers and Keywords
    if kind == 'IDENTIFIER':
        lower_value = value.lower()
        if lower_value in keywords:
            tokens_list.append(("KEYWORD", line_num, lower_value))
            total_lexemes += 1
        elif len(value) > 8:
            error_list.append(f"Lexical Error at line {line_num}:
Identifier '{value}' exceeds max length (8 chars)")

```

```

        else:
            tokens_list.append(("IDENTIFIER", line_num, value))
            total_lexemes += 1
            if value not in symbol_table:
                symbol_table[value] = line_num
        else:
            tokens_list.append((kind, line_num, value))
            total_lexemes += 1

    return total_lexemes, tokens_list, symbol_table, error_list

# Testing the Scanner
if __name__ == '__main__':
    # Sample code with perfectly formatted syntax and deliberate
    # lexical errors
    test_code = """
start
var float temp = 36.5;
if temp > 37 then
print "Fever";
finish
finish
    """

    total, tokens, sym_table, errors = healthscript_scanner(test_code)

    print("=== Phase 1 Scanner Results ===")
    print(f"Total Lexemes Found: {total}")
    print()

    print("Tokens (<Type, Line, Value>)")
    for t in tokens:
        print(f"<{t[0]}, {t[1]}, {t[2]}>")

    print()
    print("Symbol Table")
    for identifier, line in sym_table.items():
        print(f"{identifier} (first seen at line {line})")

    print()
    print("Errors")
    if not errors:
        print("No errors! Ready for Phase 2.")
    else:
        for err in errors:
            print(err)

```

This appendix ensures that the implementation is fully documented and allows reviewers to examine the internal logic of the scanner in detail.